TOPDEVCREW · PRUEBA TÉCNICA

Asistente RAG documental con ingesta asíncrona y trazabilidad de punta a punta

Ingeniero AI Full Stack Semi-Senior · Entrégala antes del 5 de noviembre de 2026 a las 23:20 UTC

Cómo funciona

- Tienes 72 horas desde que te enviamos este proyecto.
- Puedes usar IA generativa (Claude, Copilot, ChatGPT…) como quieras.
- Solo puedes entregar UNA VEZ. Al pegar el link, congelamos el último commit de tu repositorio en ese momento. Esa es la versión que evaluamos y la que revisamos en la entrevista; los commits posteriores se ignoran y no hay una segunda entrega. Asegúrate de que el repositorio esté listo antes de pegar el link.
- Sube tu trabajo a UN repositorio PÚBLICO de GitHub. Si el proyecto tiene varias partes (backend, frontend, infraestructura), van juntas en ese único repositorio (monorepo). No aceptamos repositorios privados, varios repositorios ni archivos zip.
- En la entrevista técnica vas a recorrer tu código en tu propia pantalla y defender tus decisiones. Entrega solo lo que entiendas.
- Donde algo sea ambiguo, decide y explica por qué en DECISIONS.md.

El proyecto

Faro Administraciones, una empresa ficticia que gestiona más de 300 comunidades de propietarios, tiene miles de PDFs: actas de asamblea, reglamentos internos, contratos con proveedores y facturas escaneadas. Los administradores pierden horas buscando respuestas como: '¿qué se aprobó sobre la reparación del ascensor en la asamblea de marzo?'. Debes construir un MVP de asistente documental. Un administrador sube documentos a una comunidad y la ingesta corre en segundo plano: parseo (Docling si lo consigues integrar, o un parser alternativo justificado), chunking, embeddings y almacenamiento en PostgreSQL con pgvector. Luego hace preguntas en lenguaje natural y recibe respuestas con citas verificables (documento, página y fragmento). El sistema debe aislar los datos por comunidad, decir 'no lo sé' cuando la evidencia no alcance, y dejar trazas para entender dónde se va el tiempo y el costo de cada consulta. Tienes 72 horas y puedes usar IA generativa. El núcleo se puede resolver en unas 3 horas de trabajo enfocado, y el resto del tiempo es para profundizar donde creas que aporta más. Lo que cuenta es el head commit del repositorio público en el momento en que entregues, así que cuida que ese estado sea coherente y funcione.

REQUISITOS

- API FastAPI con endpoints para crear comunidades, subir documentos (PDF y texto), consultar el estado de ingesta y hacer preguntas. Los modelos de entrada y salida van con Pydantic.
- Pipeline de ingesta asíncrono con Celery y Redis como broker: parseo, chunking, generación de embeddings e inserción en PostgreSQL con pgvector. Debe ser idempotente y reintentable, con estados visibles (pendiente, procesando, listo, fallido).
- Recuperación sobre pgvector filtrada por comunidad. Justifica la estrategia (solo vectorial o híbrida con búsqueda de texto de PostgreSQL) y el tamaño de chunk.
- Endpoint de preguntas que llame al LLM a través de una capa de abstracción (LiteLLM o equivalente) y devuelva una respuesta estructurada validada con Pydantic: respuesta, citas con documento, página y fragmento, y un indicador de suficiencia de evidencia.
- Comportamiento explícito cuando la evidencia es insuficiente o el modelo devuelve una salida inválida: reintento acotado, degradación controlada y mensaje claro al usuario, sin respuestas inventadas.

- Frontend React con subida de documentos, lista con estado de ingesta (con polling o streaming) y un chat que muestre las citas y permita abrir el fragmento fuente.
- Proveedor LLM y de embeddings intercambiable por configuración. Incluye un proveedor falso determinista para tests y desarrollo sin claves. Si usas Gemini, documenta cómo lo configuras.
- Docker Compose que levante API, worker, Redis, PostgreSQL con pgvector, frontend y un colector de OpenTelemetry con un backend de trazas sencillo.

NIVEL DE CALIDAD

- Pruebas automatizadas: unitarias para chunking y validación de respuestas, de integración para ingesta y recuperación contra PostgreSQL real, y al menos un test end-to-end con el proveedor falso. Incluye un pequeño conjunto de evaluación con preguntas y citas esperadas, y explica sus límites.
- OpenTelemetry con una traza continua desde la petición HTTP, pasando por la tarea Celery, hasta las llamadas al LLM y a la base de datos. Registra latencia, tokens y modelo usado como atributos, y logs estructurados con correlación por trace id.
- Manejo de fallos: timeouts, límites de tasa y errores del proveedor LLM, tareas Celery que mueren a mitad de camino, documentos corruptos o escaneados sin texto. Cada caso debe tener un comportamiento definido y probado.
- Seguridad: validación de tipo y tamaño de archivos, aislamiento estricto entre comunidades en cada consulta, manejo de secretos por variables de entorno y mitigación básica de prompt injection desde el contenido de los documentos.
- CI en GitHub Actions que ejecute linters, tests y construya las imágenes. Ninguna clave real debe quedar en el repositorio ni en el historial.

DECIDE Y JUSTIFICA

- ¿Cómo autenticas y autorizas a los administradores respecto de las comunidades que gestionan? Decide qué implementas y qué dejas simulado, y justifícalo en DECISIONS.md.
- Si dos documentos se contradicen (por ejemplo, un reglamento antiguo y una acta posterior), ¿cómo lo trata el sistema? Decide si lo detecta, lo muestra o lo ignora, y por qué.
- ¿Cómo cuantificas la calidad de las respuestas y de las citas con poco tiempo y sin un conjunto de datos real? Define qué mides y qué no te atreves a afirmar.
- ¿Qué haces con documentos escaneados sin capa de texto (OCR) y con documentos muy grandes? Decide el alcance del MVP y qué queda documentado como deuda.

ENTREGABLES

- Un único repositorio público de GitHub con todo el proyecto como monorepo (backend, worker, frontend e infraestructura con Docker Compose). El head commit en el momento de la entrega es la única versión que se evaluará.
- README con instrucciones para levantar todo con un solo comando, variables de entorno, cómo correr los tests y una sección honesta de qué está hecho y qué no.
- DECISIONS.md con tus decisiones sobre las preguntas abiertas, los trade-offs de arquitectura (chunking, recuperación, abstracción de LLM, estrategia de reintentos) y cómo usaste IA generativa, incluyendo qué corregiste de lo que te propuso.
- Suite de pruebas automatizadas y workflow de CI en el repositorio, con el conjunto de evaluación de preguntas y citas esperadas.
- Una captura o un breve texto con una traza de OpenTelemetry de una consulta real, comentando qué aprendiste de ella.

Entrega

Pega el link de tu repositorio público de GitHub en:

https://app.topdevcrew.com/test/b6cc434f6b6fd5518032e1ad3bf6e573